

JOÃO GUILHERME LOPES ALVES DA COSTA
JOSÉ DAVI VIANA FRANCELINO
JUSCELINO PEREIRA DE ARAUJO
THIAGO DE OLIVEIRA CORDEIRO
VICTOR BASTOS XAVIER

**COMPILADOR MINIJAVA: IMPLEMENTAÇÃO DE UM COMPILADOR
COM ANÁLISE LÉXICA, SINTÁTICA E TABELA DE SÍMBOLOS**

Relatório técnico apresentado como avaliação 1 da disciplina DIM0164 — Compiladores, ministrada pelo(a) professor(a) Valdigleis da Silva Costa, para o curso de Ciência da Computação do Departamento de Informática e Matemática Aplicada da Universidade Federal do Rio Grande do Norte - Campus Natal.

Sumário

1	Introdução	4
2	A linguagem MiniJava	4
2.1	Visão geral	4
2.2	Tipos de dados	4
2.3	Estrutura de um programa	4
2.4	Declaração de variáveis	5
2.5	Comandos	5
2.5.1	Atribuição	5
2.5.2	Condiciona (if/else)	5
2.5.3	Laço de repetição (while)	5
2.5.4	Impressão	5
2.6	Expressões e operadores	6
2.6.1	Operadores aritméticos	6
2.6.2	Operadores de comparação e lógicos	6
2.6.3	Acesso a arrays e métodos	6
2.7	Classes e herança	6
2.8	Instanciação de objetos e arrays	6
2.9	Exemplo completo: Fatorial	7
2.10	Comentários	7
3	Design da implementação	7
3.1	Etapas de implementação	7
3.2	Transformação para gramática LL(1)	8
3.3	Estrutura do projeto	9
3.4	Fluxo de compilação	9
4	Análise léxica	10
4.1	Definição dos tokens	10
4.2	Implementação com Flex	10
4.2.1	Pré-processamento integrado	10
4.2.2	Regras de reconhecimento	11
4.2.3	Tratamento de erros léxicos	11
4.3	Saída da análise léxica	11
5	Análise sintática	11
5.1	Método utilizado	11
5.2	Gramática da linguagem	11
5.3	Conjuntos FIRST	11
5.4	Tratamento de erros sintáticos	12
6	Tabela de símbolos	12
6.1	Estrutura	12
6.2	Preenchimento	12
6.3	Saída formatada	12

7	Compilação e execução	13
7.1	Pré-requisitos	13
7.2	Processo de build	13
7.3	Execução	13
7.4	Saídas possíveis	13
8	Considerações finais	13

1 Introdução

O presente relatório tem como objetivo apresentar a documentação do compilador para a linguagem *MiniJava*, desenvolvido como projeto da disciplina DIM0164 — Compiladores (2026.1, UFRN). O compilador implementa as fases de análise léxica, análise sintática e construção da tabela de símbolos.

A linguagem *MiniJava* é um subconjunto da linguagem Java, projetada para fins didáticos. O compilador foi implementado em C++ (padrão C++17) utilizando o gerador de analisadores léxicos *Flex* e um parser recursivo descendente escrito manualmente para validação da gramática LL(1).

2 A linguagem MiniJava

2.1 Visão geral

MiniJava é uma linguagem orientada a objetos com tipagem estática. Todo programa MiniJava possui obrigatoriamente uma classe principal com um método `main`, seguida de zero ou mais classes auxiliares.

2.2 Tipos de dados

A linguagem suporta os seguintes tipos:

- `int` — inteiros;
- `int[]` — arrays de inteiros;
- `boolean` — valores lógicos (`true` ou `false`);
- Tipos definidos pelo usuário (nomes de classes).

2.3 Estrutura de um programa

Todo programa MiniJava segue esta estrutura:

Listing 1: Estrutura básica de um programa MiniJava

```
1 class NomePrincipal {  
2     public static void main(String[] args) {  
3         // comandos do programa principal  
4     }  
5 }  
6  
7 class OutraClasse {  
8     // variaveis de instancia  
9     int x;  
10  
11     // metodos  
12     public int metodo(int param) {  
13         // corpo  
14         return valor;  
15     }
```

16 }

2.4 Declaração de variáveis

Variáveis são declaradas com tipo seguido de nome e ponto-e-vírgula:

Listing 2: Declaração de variáveis

```
1 int x;  
2 int[] numeros;  
3 boolean flag;  
4 MinhaClasse obj;
```

2.5 Comandos

2.5.1 Atribuição

Listing 3: Atribuições

```
1 x = 10;  
2 numeros[0] = 42;
```

2.5.2 Condicional (if/else)

O if em MiniJava sempre exige um bloco else:

Listing 4: Estrutura condicional

```
1 if (x < 10)  
2     y = 1;  
3 else  
4     y = 2;
```

2.5.3 Laço de repetição (while)

Listing 5: Laço while

```
1 while (i < n) {  
2     System.out.println(i);  
3     i = i + 1;  
4 }
```

2.5.4 Impressão

A única forma de saída é `System.out.println`, que imprime um inteiro:

Listing 6: Impressão de valores

```
1 System.out.println(42);  
2 System.out.println(x + y);
```

2.6 Expressões e operadores

2.6.1 Operadores aritméticos

- + (adição), - (subtração), * (multiplicação)

2.6.2 Operadores de comparação e lógicos

- < (menor que), > (maior que)
- && (E lógico), ! (negação)

2.6.3 Acesso a arrays e métodos

Listing 7: Expressões com arrays e métodos

```
1 // Acesso a array
2 x = numeros[i];
3
4 // Tamanho do array
5 tam = numeros.length;
6
7 // Chamada de metodo
8 resultado = obj.metodo(arg1, arg2);
```

2.7 Classes e herança

Classes podem herdar de outra classe usando **extends**:

Listing 8: Herança de classes

```
1 class Animal {
2     int idade;
3
4     public int getIdade() {
5         return idade;
6     }
7 }
8
9 class Cachorro extends Animal {
10     int peso;
11
12     public int getPeso() {
13         return peso;
14     }
15 }
```

2.8 Instanciação de objetos e arrays

Listing 9: Criação de objetos e arrays

```
1 // Novo objeto
```

```

2 obj = new MinhaClasse();
3
4 // Novo array de inteiros com 10 posicoes
5 numeros = new int[10];

```

2.9 Exemplo completo: Fatorial

Listing 10: Programa que calcula o fatorial de 10

```

1 class Factorial {
2     public static void main(String[] a) {
3         System.out.println(new Fac().ComputeFac(10));
4     }
5 }
6
7 class Fac {
8     public int ComputeFac(int num) {
9         int num_aux;
10        if (num < 1)
11            num_aux = 1;
12        else
13            num_aux = num * (this.ComputeFac(num - 1));
14        return num_aux;
15    }
16 }

```

Neste exemplo:

- A classe `Factorial` contém o `main`, que instancia `Fac` e chama `ComputeFac`;
- A classe `Fac` define um método recursivo que calcula o fatorial;
- `this.ComputeFac(num - 1)` é uma chamada recursiva ao próprio método.

2.10 Comentários

A linguagem suporta dois estilos de comentários:

Listing 11: Comentários

```

1 // Comentario de linha unica
2
3 /* Comentario de
4    multiplas linhas */

```

3 *Design* da implementação

3.1 Etapas de implementação

O desenvolvimento do compilador seguiu as seguintes etapas:

1. **Transformação da gramática para LL(1):** a gramática original da linguagem MiniJava foi transformada para eliminar recursão à esquerda e fatorar produções ambíguas, resultando em uma gramática LL(1) adequada para um parser recursivo descendente;
2. **Implementação do analisador léxico:** definição dos tokens e construção do scanner com *Flex*;
3. **Implementação do analisador sintático:** construção do parser recursivo descendente a partir da gramática LL(1);
4. **Construção da tabela de símbolos:** integração ao parser para registrar declarações durante o parsing.

3.2 Transformação para gramática LL(1)

A gramática original da linguagem MiniJava possui construções que não são diretamente compatíveis com parsers LL(1), como produções com prefixos comuns. Para resolver isso, foram aplicadas as seguintes técnicas:

- **Fatoração à esquerda:** produções como `Cmd`, que podem começar com `Id` tanto para atribuição simples quanto para atribuição em array, foram fatoradas em `Cmd` e `Cmd'`;
- **Eliminação de recursão à esquerda:** expressões que tinham recursão à esquerda (como operações binárias encadeadas) foram transformadas usando o padrão `Exp → PrimExp Exp'` com `Exp'` recursivo à direita;
- **Separação de alternativas:** produções como `DefCl'` separam o caso com herança (`extends`) do caso sem.

A gramática LL(1) resultante é apresentada a seguir:

Prog	→ MainC DefCl
MainC	→ 'class' Id '{' 'public' 'static' 'void' 'main' '(' 'String' '[' ']' Id ')' '{' Cmd '}' '}'
DefCl	→ 'class' Id DefCl' λ
DefCl'	→ '{' DefVar DefMet '}' DefCl 'extends' Id '{' DefVar DefMet '}' DefCl
DefVar	→ Type Id ';' DefVar λ
DefMet	→ 'public' Type Id '(' DefMetArgs ')' ' ' '{' DefVar Cmd 'return' Exp ';' '}' DefMet λ
DefMetArgs	→ Args λ
Type	→ 'int' '[' ']' 'boolean' 'int' Id
Args	→ Type Id Args'
Args'	→ ',' Args λ

```

Cmd      → '{' CmdList '}'
         | 'if' '(' Exp ')' Cmd 'else' Cmd
         | 'while' '(' Exp ')' Cmd
         | 'System' '.' 'out' '.' 'println' '(' Exp ')' ';'
         | Id Cmd'

Cmd'     → '=' Exp ';'
         | '[' Exp ']' '=' Exp ';'

CmdList  → Cmd CmdList | λ

Exp      → PrimExp Exp'
Exp'     → Op PrimExp Exp'
         | '[' Exp ']' Exp'
         | '.' PostExpDot
         | λ

Op       → '&&' | '<' | '>' | '+' | '-' | '*'

PostExpDot → 'length' Exp'
           | Id '(' ListExp ')' Exp'

ListExp  → Exp ListExp' | λ
ListExp' → ',' Exp ListExp' | λ

PrimExp  → 'new' PrimExp' | '!' Exp | '(' Exp ')'
         | 'true' | 'false' | Id | Number | 'this'
PrimExp' → Id '(' ')' | 'int' '[' Exp ']'

```

Esta gramática garante que, para cada não-terminal, o token atual (lookahead) é suficiente para determinar univocamente qual produção aplicar.

3.3 Estrutura do projeto

```

src/
  token.h          - Definicao dos tipos de token
  lexer.l          - Analisador lexico (Flex)
  parser.h         - Analisador sintatico
  symbol_table.h   - Tabela de simbolos
  main.cpp         - Programa principal
assets/            - Programas de teste (.ling)
Makefile          - Automacao de build

```

3.4 Fluxo de compilação

O fluxo de dados entre os módulos segue a seguinte sequência:

1. O `main.cpp` abre o arquivo fonte (`.ling`) e configura o *Flex*;
2. O *Flex* (`lexer.l`) percorre o código fonte caractere a caractere, gerando uma lista de tokens;

3. O `main.cpp` adiciona um token sentinela `END_OF_FILE` ao final da lista;
4. O `Parser` (`parser.h`) consome a lista de tokens, valida a gramática e preenche a tabela de símbolos;
5. O resultado é exibido: sucesso com tabela de símbolos, ou mensagem de erro.

4 Análise léxica

4.1 Definição dos tokens

Os tokens são definidos no arquivo `token.h` através da enumeração `TokenType`, organizada em 5 categorias:

- **Palavras-chave** (21 tokens): `class`, `public`, `static`, `void`, `main`, `String`, `return`, `int`, `boolean`, `if`, `else`, `while`, `System`, `out`, `println`, `true`, `false`, `this`, `new`, `extends`, `length`;
- **Operadores** (8 tokens): `&&`, `<`, `>`, `+`, `-`, `*`, `=`, `!`;
- **Delimitadores** (9 tokens): `(`, `)`, `[`, `]`, `{`, `}`, `;`, `,`, `.`;
- **Literais e identificadores**: `ID` (nomes) e `NUMBER` (inteiros);
- **Controle**: `END_OF_FILE` e `UNKNOWN`.

Cada token é representado pela estrutura:

```

1 struct Token {
2     TokenType type;      // Categoria do token
3     std::string lexeme;  // Texto original no código fonte
4     int line;           // Linha onde foi encontrado
5 };

```

4.2 Implementação com Flex

O analisador léxico é implementado no arquivo `lexer.l` utilizando o gerador *Flex*. O *Flex* gera um autômato finito determinístico (AFD) a partir de regras de expressões regulares.

4.2.1 Pré-processamento integrado

O `lexer` atua simultaneamente como pré-processador, removendo comentários (de linha e de bloco) e espaços em branco.

4.2.2 Regras de reconhecimento

As definições de expressões regulares reutilizáveis são:

DIGIT	[0-9]
LETTER	[a-zA-Z]
ID	{LETTER}({LETTER} {DIGIT} "_")*
NUMBER	{DIGIT}+

A prioridade das regras segue a ordem de aparição no arquivo: comentários, whitespace, palavras-chave, operadores, delimitadores, identificadores, números e erros.

4.2.3 Tratamento de erros léxicos

Quando o lexer encontra um token inválido, ele calcula a distância de Levenshtein entre o token e todas as palavras-chave. Se a distância for menor ou igual a 2, sugere a correção mais provável:

```
Erro lexico na linha 3: token invalido "clas"
Sugestao: voce quis dizer "class"?
```

4.3 Saída da análise léxica

A lista de tokens pode ser visualizada com a flag `-tokens`:

```
$ ./build/compiler --tokens assets/prog-factorial.ling
<CLASS, "class", line 1>
<ID, "Factorial", line 1>
<LBRACE, "{", line 1>
<PUBLIC, "public", line 2>
...
```

5 Análise sintática

5.1 Método utilizado

O analisador sintático é implementado como um *parser* recursivo descendente no arquivo `parser.h`. Cada regra da gramática LL(1) corresponde a uma função no código. O parser utiliza *lookahead* de 1 token para decidir qual produção seguir.

5.2 Gramática da linguagem

A gramática LL(1) completa utilizada pelo parser foi apresentada na Seção 3.2. Cada função do parser corresponde diretamente a uma produção dessa gramática.

5.3 Conjuntos FIRST

Para decidir qual produção seguir, o parser verifica os conjuntos FIRST:

- $\text{FIRST}(\text{Type}) = \{ \text{int}, \text{boolean}, \text{ID} \}$

- $\text{FIRST}(\text{Cmd}) = \{ \{, \text{if}, \text{while}, \text{System}, \text{ID} \}$
- $\text{FIRST}(\text{Op}) = \{ \&\&, <, >, +, -, * \}$
- $\text{FIRST}(\text{PrimExp}) = \{ \text{new}, !, (, \text{true}, \text{false}, \text{ID}, \text{NUMBER}, \text{this} \}$

5.4 Tratamento de erros sintáticos

Ao encontrar um token inesperado, o parser reporta a linha, o token esperado e o encontrado:

Erro sintatico na linha 5: esperado SEMICOLON mas encontrou RBRACE

6 Tabela de símbolos

6.1 Estrutura

A tabela de símbolos é implementada no arquivo `symbol_table.h` como um vetor sequencial. Cada entrada contém:

- `name` — nome do identificador;
- `type` — tipo associado;
- `scope` — escopo de declaração;
- `line` — linha no código fonte;
- `category` — classificação (`CLASS`, `METHOD`, `INSTANCE_VAR`, `LOCAL_VAR`, `PARAMETER`).

6.2 Preenchimento

A tabela é preenchida durante a análise sintática. Sempre que o parser reconhece uma declaração, insere um registro com o escopo apropriado:

- Classe $\rightarrow$ escopo `global`
- Método $\rightarrow$ escopo é o nome da classe
- Variável local/parâmetro $\rightarrow$ escopo é `Classe.metodo`

6.3 Saída formatada

Ao final de uma análise bem-sucedida, a tabela é impressa:

```
=== Tabela de Simbolos ===
Nome          Tipo          Escopo          Linha  Categoria
-----
Factorial     class         global          1      classe
main          void          Factorial       2      metodo
args          String[]      Factorial.main  2      parametro
```

Fac	class	global	7	classe
ComputeFac	int	Fac	9	metodo
num	int	Fac.ComputeFac	9	parametro
num_aux	int	Fac.ComputeFac	10	var. local

7 Compilação e execução

7.1 Pré-requisitos

- g++ (C++17)
- flex
- make

Instalação no Ubuntu/WSL:

```
sudo apt install build-essential flex
```

7.2 Processo de build

```
make all      # Compila o projeto
make clean    # Remove artefatos de build
```

7.3 Execução

```
./build/compiler programa.ling      # Analise completa
./build/compiler --tokens programa.ling  # Apenas tokens
```

7.4 Saídas possíveis

- **Sucesso:** mensagem de confirmação seguida da tabela de símbolos;
- **Erro léxico:** mensagem com linha e sugestão de correção;
- **Erro sintático:** mensagem com linha, token esperado e token encontrado.

8 Considerações finais

O compilador implementado cobre as três primeiras fases de um compilador tradicional: análise léxica, análise sintática e construção da tabela de símbolos. A integração entre o *Flex* e um parser recursivo descendente manual permite controle total sobre as mensagens de erro e a construção incremental da tabela de símbolos.

Como trabalhos futuros, o compilador pode ser estendido com:

- Análise semântica (verificação de tipos, variáveis não declaradas);
- Geração de código intermediário;
- Otimização e geração de código alvo.